

Relatório Final

Classificador de Áudio Utilizando Perceptron em FPGA

Carlos Eduardo Obristo

Junho de 2026

1 Objetivo e Funcionamento do Projeto

O objetivo deste projeto foi implementar, utilizando VHDL e uma FPGA Intel MAX10 (DE10-Lite), um sistema capaz de classificar pequenos trechos de áudio nas classes **grave** ou **agudo**. O sistema utiliza um neurônio artificial do tipo Perceptron previamente treinado em software, realizando apenas a inferência em hardware.

O funcionamento do sistema é dividido em quatro etapas principais:

1. Seleção da amostra através das chaves da placa;
2. Leitura de 128 amostras armazenadas em uma memória ROM;
3. Extração das características Energia e Zero Crossing Rate (ZCR);
4. Classificação através do Perceptron.

A Figura 1 apresenta a arquitetura geral do sistema.

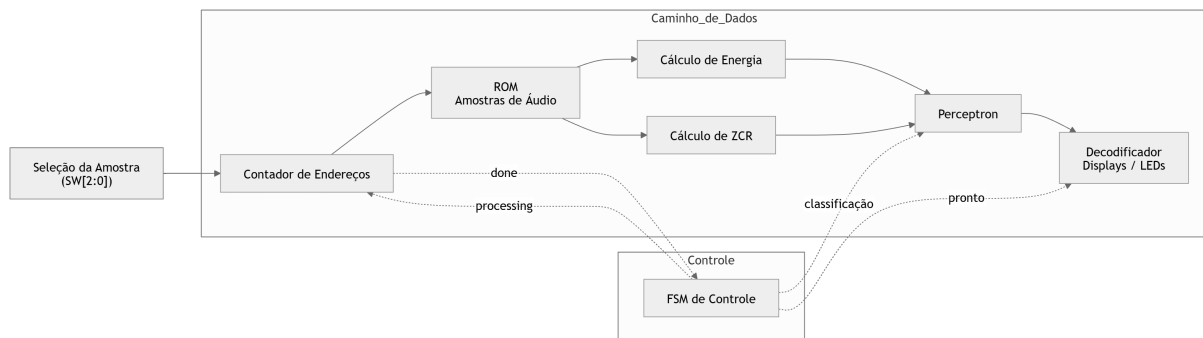


Figura 1: Arquitetura geral do sistema.

Todo o funcionamento é controlado por uma Máquina de Estados Finitos (FSM), responsável por iniciar a leitura da ROM, aguardar o término do processamento e habilitar a classificação do Perceptron.

2 Elementos Funcionais Desenvolvidos

O projeto foi dividido em módulos independentes, facilitando o desenvolvimento e os testes individuais.

- **Address Counter:** gera os endereços da memória ROM de acordo com a amostra selecionada.
- **ROM:** armazena seis sinais de áudio previamente convertidos para o formato `.mif`.
- **Energy Calculator:** calcula a energia do sinal através da soma dos quadrados das amostras.
- **ZCR Calculator:** calcula a quantidade de cruzamentos por zero do sinal.
- **Neuron:** implementa um Perceptron utilizando pesos e bias obtidos previamente em Python.
- **Controller FSM:** controla toda a sequência de execução do sistema.

O treinamento do Perceptron foi realizado em Python utilizando as mesmas características extraídas em hardware. Os pesos obtidos foram posteriormente incorporados ao código VHDL para realizar apenas a inferência na FPGA.

Durante o desenvolvimento foi utilizada uma ferramenta de Inteligência Artificial Generativa como apoio para esclarecimento de dúvidas, depuração do código VHDL e discussão de alternativas de implementação. Toda a arquitetura do projeto, decisões de projeto, testes e integração dos módulos foram realizados pelo autor.

3 Geração de Amostras e Treinamento do Classificador

Para viabilizar a validação e o treinamento do sistema, foi desenvolvido um ecossistema em Python para a geração sintética do conjunto de dados e o treinamento do modelo de aprendizado de máquina.

3.1 Geração Sintética das Amostras de Áudio

Utilizando o script `generate_dataset.py`, foram geradas 100 amostras de áudio (50 graves e 50 agudas) com duração de 128 pontos discretos cada. Cada ponto t (de 0 a 127) foi sintetizado por meio de funções senoidais seguindo a equação:

$$x[t] = A \cdot \sin\left(\frac{2\pi \cdot f \cdot t}{128} + \phi\right) + \text{ruído} \quad (1)$$

Os parâmetros foram selecionados de forma aleatória dentro de intervalos específicos para garantir variedade estatística e robustez ao modelo:

- **Frequência (f):** Parâmetro diferenciador das classes. Para amostras **graves**, $f \in [1.0, 4.0]$ (sinal de baixa frequência). Para amostras **agudas**, $f \in [15.0, 30.0]$ (sinal de alta frequência).
- **Amplitude (A):** Sorteada de forma inteira entre 40 e 120.

- **Fase (ϕ):** Deslocada aleatoriamente no intervalo $[0, 2\pi]$ radianos.
- **Ruído:** Adição de ruído uniforme aleatório na faixa de $[-5, 5]$ em cada ponto.

Após a síntese, os valores foram saturados na faixa limite de $[-128, 127]$ para representar valores de 8 bits com sinal (**signed**), compatíveis com o formato físico da memória ROM do projeto.

3.2 Treinamento do Perceptron

A extração de características das amostras geradas foi efetuada pelo script `extract_features.py`, calculando a Energia ($E = \sum x[n]^2$) e o número de cruzamentos por zero (ZCR).

O treinamento do classificador foi realizado com o script `train_perceptron.py`. Para equiparar a ordem de grandeza da Energia (máximo teórico de $\approx 2 \times 10^6$) com a do ZCR (máximo de 128), a característica da energia foi normalizada por meio de um deslocamento lógico de 14 bits para a direita (divisão inteira por 16.384), facilitando o processamento aritmético simples no hardware:

$$E_{\text{escalada}} = \lfloor E/16384 \rfloor \quad (2)$$

O modelo de Perceptron simples foi treinado usando descida de gradiente estocástica por 1000 épocas. O algoritmo convergiu com sucesso, encontrando os seguintes parâmetros inteiros ótimos:

- Peso para Energia (w_1): -63
- Peso para ZCR (w_2): 95
- Viés (Bias): -507

Esses coeficientes foram importados como constantes inteiras no arquivo VHDL `neuron.vhd`, de modo que a inferência do neurônio em hardware executa exatamente a mesma fronteira de decisão de ponto fixo treinada em software.

4 Resultados

Os módulos foram inicialmente testados individualmente por simulação funcional. Após a validação, o sistema completo foi implementado na FPGA.

A Figura 2 apresenta uma captura realizada utilizando o SignalTap durante a execução do sistema.

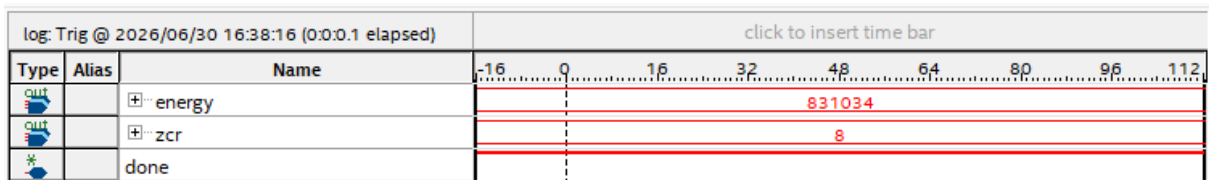


Figura 2: Captura do SignalTap durante a classificação de uma amostra.

Os valores calculados para Energia e ZCR coincidiram com aqueles obtidos em Python durante a etapa de treinamento, permitindo que o Perceptron realizasse corretamente a classificação das amostras utilizadas nos testes.

5 Conclusão

O desenvolvimento do projeto permitiu aplicar diversos conceitos apresentados na disciplina, principalmente descrição de hardware em VHDL, máquinas de estados, circuitos sequenciais, organização modular de projetos e utilização de memória ROM em FPGA.

A principal dificuldade encontrada foi sincronizar corretamente a leitura da memória ROM com os módulos responsáveis pelo cálculo das características, devido à latência da memória e ao sincronismo entre os diferentes blocos do circuito.

Além dos conteúdos da disciplina, foi necessário estudar conceitos de processamento digital de sinais, especialmente Energia e Zero Crossing Rate, além do funcionamento básico de um Perceptron e da implementação de operações aritméticas em hardware utilizando VHDL.

A utilização da Inteligência Artificial contribuiu principalmente para acelerar a depuração do projeto e discutir alternativas de implementação, mas todas as decisões de arquitetura, integração entre módulos, validação dos resultados e testes finais foram realizadas pelo autor.